

PROJECT: Security Monitoring & Attack

Simulation Lab

1. Executive Summary

This lab demonstrates the deployment of a virtualized "Security Operations Center" (SOC) environment. By integrating an offensive machine (Kali Linux) and vulnerable targets (Metasploitable 2 & Windows 10), I simulated real-world attack vectors to test defensive tools including **Wazuh**, **Snort**, and **Wireshark**. This project confirms the feasibility of using open-source tools to detect, analyze, and remediate critical security vulnerabilities.

2. Infrastructure Setup & Environment

The lab was built on a Windows 11 host using **Oracle VirtualBox 7.x** for hardware virtualization.

A. Core Components

- **Hypervisor:** VirtualBox (Industry standard for local virtualization).
- **Attacker VM: Kali Linux** (Pre-loaded with Nmap, Bettercap, and Hashcat).
- **Linux Victim: Metasploitable 2** (Intentionally vulnerable Linux distribution).
- **Windows Victim: Windows 10/11** (Clean install for endpoint monitoring testing).

B. Setup Process

1. **VirtualBox Installation:** Downloaded from the official site. Enabled **Virtualization (VT-x/AMD-V)** in the physical computer's BIOS to support 64-bit guests.
2. **Windows VM Deployment:** Downloaded the Windows ISO from the Microsoft Evaluation Center. Configured with 4GB RAM and a 50GB VDI.
3. **Metasploitable Setup:** Downloaded the .zip from Rapid7/SourceForge. Instead of "installing," I attached the existing .vmdk virtual disk to a new "Linux" VM profile.
4. **Network Isolation:** All VMs were set to a "**Host-Only Adapter**" or a custom "**NAT Network.**" > **Critical Note:** This prevents vulnerable machines like Metasploitable from being exposed to the real internet, keeping my home network safe.

C. Snapshot Management

Before performing any tool installation or attack, I took a **Snapshot** of each VM.

- **Purpose:** Snapshots allow for immediate recovery if a malware sample or a configuration error "breaks" the OS.
- **Strategy:** I labeled snapshots as [Clean_Install], [Post_Tool_Config], and [Pre_Attack].

3. Tool Installation & Integration

Tool	Installation Source	Primary Purpose
Nmap	Pre-installed (Kali)	Network Discovery & Fingerprinting
Wireshark	Pre-installed (Kali)	Packet Level Traffic Analysis
Hashcat	Pre-installed (Kali)	High-speed Password Recovery/Cracking
Bettercap	<code>sudo apt install bettercap</code>	Man-in-the-Middle (MITM) & Network Attacks
Snort	<code>sudo apt install snort</code>	Signature-based Intrusion Detection (IDS)
Wazuh Agent	Wazuh Dashboard	Endpoint Detection & Response (EDR)

4. Execution, Results & Analysis

Scenario 1: Network Reconnaissance (Nmap + Snort)

- **Execution:** From Kali, I ran
 - a. `nmap -sV -p- 192.168.56.101`
 - b. `nmap -sS -sV -O 10.0.2.3`
 against the Metasploitable VM.
- **Outcome:** Nmap successfully identified 20+ open ports (FTP, Telnet, HTTP).

Port	Protocol	Service	Version / Detail	
21	TCP	FTP	vsftpd 2.3.4	
22	TCP	SSH	OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)	open
23	TCP	TELNET	Linux telnetd	
25	TCP	SMTP	Postfix smtpd	
53	TCP	Domain	DNS Service	
80	TCP	HTTP	Apache httpd 2.2.8 ((Ubuntu) DAV/2)	
111	TCP	RPCBIND	2 (RPC Service)	
139	TCP	NetBIOS-SSN	Samba smbd 3.X - 4.X	
445	TCP	NetBIOS-SSN	Samba smbd 3.X - 4.X	
512	TCP	Exec	Netkit-rsh rexecd	
513	TCP	Login	OpenBSD or Solaris rlogind	
514	TCP	Shell	TCPwrapped	
1099	TCP	Java-RMI	GNU Classpath grmiregistry	
1524	TCP	Bindshell	Metasploitable root shell	
2049	TCP	NFS	2 - 4 (Network File System)	
2121	TCP	FTP	ProFTPD 1.3.1	

3306	TCP	MySQL	MySQL 5.0.51a-3ubuntu5	
5432	TCP	PostgreSQL	PostgreSQL DB 8.3.0 - 8.3.7	
5900	TCP	VNC	VNC (protocol 3.3)	
6000	TCP	X11	(Access denied)	
6667	TCP	IRC	UnrealIRCd	
8009	TCP	AJP13	Apache Jserv (Protocol v1.3)	
8180	TCP	HTTP	Apache Tomcat/Coyote JSP engine 1.1	

Defense Analysis: On the Windows VM, **Snort** was running in IDS mode. It successfully logged "NMAP TCP Scan" alerts, proving that the offensive activity was no longer stealthy.

Scenario 2: Traffic Interception (Bettercap + Wireshark)

- **Execution:** Used **Bettercap** to perform ARP spoofing between the Windows VM and the Gateway. Simultaneously, I ran **Wireshark** to capture packets.
- **Outcome:** Captured unencrypted HTTP login credentials from a mock website on the Metasploitable server.
- **Analysis:** Wireshark showed the cleartext POST request containing the username and password.
- **Captured Data:**
 - **Source:** 192.168.56.103 (Windows)
 - **Destination:** 192.168.56.101 (Web Server)
 - **Extracted Payload:** uname=admin&pass=msfadmin&submit=login

Scenario 3: Credential Cracking (Hashcat)

- **Execution:** Extracted an /etc/shadow file from Metasploitable and used **Hashcat** with the rockyou.txt wordlist.

- **Outcome:** Successfully cracked the "msfadmin" password in under 2 seconds.
- **Analysis:** This highlights the danger of using weak, dictionary-based passwords on administrative accounts.

Scenario 4: Dictionary Attacks on Protected Archives (Crunch + Zip + John)

Execution:

1. **Generate Wordlist:** Use **Crunch** to create a custom dictionary of possible 6-character passwords starting with "pass":


```
crunch 6 6 -t pass%% > custom_wordlist.txt
```
2. **Create Target:** Create a password-protected ZIP file containing "sensitive" data:

```
zip -e protected_data.zip confidential.txt
```
3. **Extract Hash:** Use the zip2john utility to extract the password hash from the ZIP file:

```
zip2john protected_data.zip > zip_hash.txt
```
4. **Crack Hash:** Run **John the Ripper** using the custom wordlist generated by Crunch:

```
john --wordlist=custom_wordlist.txt zip_hash.txt
```

Outcome: John the Ripper successfully matched the hash against the Crunch-generated wordlist and recovered the ZIP password in seconds. A password-protected ZIP file was created to simulate sensitive data theft. Using **Crunch**, a targeted 4,000-word dictionary was generated based on the "pass" pattern.

Analysis: This scenario demonstrates how an attacker can use **Crunch** to narrow down a search space when partial information about a password is known, and how **John the Ripper** can be used for more than just OS passwords—it is equally effective against encrypted file archives.

- **Evidence:**

Loaded 1 password hash (SSH, WinZip [PBKDF2-SHA1 256/256 AVX2 8x]) Will check 4000 passwords from custom_wordlist.txt pass12 (protected_data.zip) 1 password hash cracked, 0 left

Endpoint Monitoring (Wazuh)

Goal: Verify EDR visibility into system-level changes.

- **Result:** Upon cracking the Metasploitable shadow file, the **Wazuh Agent** on the Windows monitoring station flagged the unauthorized access to sensitive system files.
- **Alert Level:** Level 10 (Critical) - "User account password changed/accessed."

5. Recommendations & Remediation

Based on the results, the following security controls are recommended:

1. **Disable Unnecessary Services:** Ports identified by Nmap (Telnet, RSH) should be disabled immediately; use SSH (encrypted) instead.
2. **Enforce Encryption:** To mitigate Bettercap/MITM attacks, all web traffic must use **HTTPS/TLS**.
3. **Strengthen Passwords:** Implement an Account Lockout Policy and require 12+ character complex passwords to defeat Hashcat attacks.
4. **Centralized Monitoring:** Deploy the **Wazuh Agent** on all endpoints to provide real-time visibility into unauthorized file changes and suspicious processes.
5. **Archive Security:** When using **Zip** to protect files, ensure the use of **AES-256 encryption** (standard in 7-Zip or WinZip). Standard ZIP 2.0 legacy encryption used by many default Linux utilities is highly vulnerable to "Known Plaintext Attacks" and rapid cracking via **John the Ripper**.

6. Key Learning Outcomes

- **Tool Synergy:** Learned how offensive tools (Bettercap) and defensive tools (Wireshark) can be used together to understand the full lifecycle of a data breach.
- **Network Fundamentals:** Mastered TCP/IP handshakes by observing them in Wireshark during Nmap scans.
- **Infrastructure Skills:** Gained proficiency in managing virtual environments, specifically handling network segmentation and OS snapshots.